

BACHELOR'S THESIS

Exploring Hyperbolic Embeddings of Finite Metric Spaces

Raphael Vogt

Submitted: April 22, 2024. Student Number: 4133416

Supervisor: Prof. Dr. Christoph Schnörr

Faculty of Mathematics and Computer Science
Ruprecht-Karls-Universität Heidelberg

I hereby declare that I wrote the present bachelor thesis independently under supervision and that no other sources or learning aids, other than those listed, have been used.

I have indicated all parts of the work in which sources are used according to their wording or to their meaning. I also hereby declare that my thesis has not been prepared for another examination or assignment, either in its entirety or excerpts thereof.

I am aware of the fact that violations of copyright has legal consequences.

Place, Date

Raphael Vogt

Abstract

In order to achieve good results in machine learning applications, it is first necessary to find suitable representations of the data. This bachelor thesis presents an approach how data with a graph structure can be embedded in a model of hyperbolic space, the Poincaré disk. For this purpose, the data is first initialized in the Poincaré disk, for which the thesis presents various methods. In the following, the initialized data points are optimized by a version of stochastic gradient descent modified for Riemannian manifolds. Finally, experiments show, that this method delivers good results for smaller, simpler graphs. To apply it to real-world data, however, parallelization to improve the runtime, as well as adjustments to the efficiency of the optimization to achieve convergence still need to be made.

Zusammenfassung

Um gute Ergebnisse in Machine Learning Anwendungen zu erzielen, ist es zunächst notwendig, passende Repräsentationen der Daten zu finden. Diese Bachelorarbeit präsentiert einen Ansatz, wie Daten, die eine Graphenstruktur aufweisen, dazu in ein Modell des hyperbolischen Raumes, die Poincaré Disk, eingebettet werden können. Dafür werden die Daten erst in die Poincaré Disk initialisiert, wofür die Arbeit verschiedene Methoden vorstellt. Die initialisierten Daten werden dann durch eine für Riemannsche Mannigfaltigkeiten abgewandelte Version von Stochastic Gradient Descent optimiert. In Experimenten wird zum Schluss gezeigt, dass dieses Verfahren für kleinere simplere Graphen auch bereits gute Ergebnisse liefert. Um es auf Daten aus der echten Welt anzuwenden, müssen allerdings noch Parallelisierung zur Verbesserung der Laufzeit sowie Anpassungen an der Effizienz der Optimierung zum Erreichen von Konvergenz vorgenommen werden.

Contents

1. Introduction	1
1.1. Overview and Motivation	1
1.2. Organization	1
2. Background	3
2.1. Riemannian Geometry	3
2.1.1. Tangent Space	3
2.1.2. Lengths and Metrics	4
2.1.3. The Exponential Map	4
2.2. Numerical Optimization on Manifolds	5
2.2.1. First Order Optimization	5
2.2.2. Riemannian Stochastic Gradient Descent	6
2.2.3. Line Search - Accelerating RSGD	7
3. Poincaré Disk and Metric Data Embeddings	8
3.1. Hyperbolic Geometry	8
3.1.1. The Poincaré Disk	8
3.1.2. Riemannian distances in the Poincaré Disk	9
3.2. Embedding Metric Data into the Poincaré Disk	11
3.2.1. Graph Data	11
3.2.2. Loss Functions	12
3.2.3. Mini-Batch RSGD	13
3.2.4. Initialization Strategies	14
4. Experiments and Discussion	17
4.1. Implementation Details	17
4.2. Evaluation of Toy Data	17
4.2.1. Convergence Results of the Embedding Process	17
4.2.2. Runtime Efficiency of the Embedding Process	19
4.2.3. Comparing different Batch Sizes	20

4.3. Conclusion	21
4.4. Visualization of Exemplary Embeddings	22
Appendices	26
A. Euclidean Gradient of Loss Functions	27
B. Partial Derivative of Riemannian Distance	29

1. Introduction

1.1. Overview and Motivation

A lot of data which is nowadays used in real world machine learning applications, e.g. words in a lexical database [Mil94], is represented with a graph like structure. Before those data gets employed in various machine learning tasks, it's indispensable to find a suitable representation of the data to improve the performance of the machine learning model [BCV12]. Therefore, the thesis discusses a space in which graphs can be well represented, which is henceforth called the embedding space.

The most fundamental embedding space is the Euclidean vector space. However, more complex graphs in particular are difficult to embed in the Euclidean vector space, since a high dimensionality is required to represent the complex relationships in the graph well [NJT14], which in turn is computationally unacceptable. This thesis shows how a model of hyperbolic space, the Poincaré disk to be precise, can be used as an embedding space instead to tackle this problem. It brings the advantage, that due to the hyperbolic structure of the space, good representations of graphs can be found even with low dimensions in the embedding space [NK17].

The thesis differs from the procedure proposed in [NK17], in that the optimization in the embedding space follows a full Riemannian approach, in which the **exponential map** is used instead of a retraction to compute updated data representations during the optimization. In addition, advanced **initialization methods** are also discussed which can significantly improve the performance of the embeddings and instead of a fixed learning rate during the optimization, an **adaptive learning rate** is used.

1.2. Organization

The thesis is organized as follows. In the beginning, **Chapter 2** gives an overview of the field of Riemannian geometry and numerical optimization on manifolds. This is important because it lays the foundation for the embedding space and shows how it can be used in the context of numerical optimization. **Chapter 3** introduces the Poincaré

disk, which is the model of the hyperbolic space that will be used as embedding space from now on. In addition, the embedding process is described in more detail and it is shown how it can be further improved with adjustments in the optimization and better initializations. Finally, [Chapter 4](#) shows how the presented method is applied to toy data and debates the results obtained. This is rounded off with a conclusion in which the entire procedure is evaluated and possible weaknesses and improvements are debated.

2. Background

2.1. Riemannian Geometry

In order to gain an understanding of the embedding space, the basics of Riemannian geometry are now explained.

2.1.1. Tangent Space

In the following a basic understanding of smooth manifolds is assumed. This section follows the introduction of tangent spaces $T_p M$, $p \in M$ in Chapter 1 of [Car92]. At every point p of an arbitrary smooth manifold M , tangent vectors can be defined. For this, $\alpha : (-\epsilon, \epsilon) \rightarrow M$ denotes a smooth curve which holds $\alpha(0) = p \in M$, i.e. it evaluates to p at $t = 0$. Additionally $\mathcal{D}$ refers to the set of functions, which are differentiable at point p . A tangent vector w at p is the tangent vector to $\dot{\alpha}(0)$ defined as

$$\dot{\alpha}(0)f = \left. \frac{d(f \circ \alpha)}{dt} \right|_{t=0}, \quad f \in \mathcal{D}. \quad (2.1)$$

The set of all tangent vectors spans the tangent space $T_p M$ at point p . Geometrically this space can be interpreted as the linear approximation of M at p .

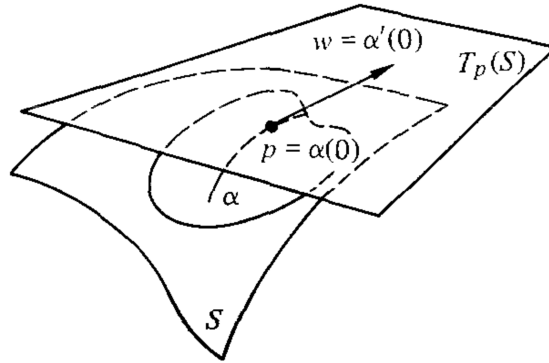


Figure 2.1.: The tangent space of a surface S at point p [Car16].

2.1.2. Lengths and Metrics

While optimizing the embeddings, distances are computed in both the Euclidean and the hyperbolic space. In the Euclidean case, this is trivial using the Euclidean metric, due to the flat space. In the hyperbolic case on the other hand, it's necessary to use some Riemannian metric. A Riemannian metric associates a smoothly varying symmetric and positive-definite bilinear form (inner product)

$$g(\cdot, \cdot) = \langle \cdot, \cdot \rangle_p : T_p M \times T_p M \rightarrow \mathbb{R} \quad (2.2)$$

to each point p on M [AMS09]. In practice, ds is also often used to denote the Riemannian metric. The Poincaré's Disk model, which is used for the embedding space, can be considered a Riemannian manifold, which is a smooth manifold with an associated Riemannian metric [Car92]. Using the Riemannian metric of the manifold the length

$$L(\gamma) = \int_{t_0}^{t_1} \sqrt{g(\dot{\gamma}, \dot{\gamma})} dt \quad (2.3)$$

of a curve $\gamma : [t_0, t_1] \rightarrow M$ can be computed [AMS09]. The Riemannian distance applies this, to denote the distance between two points $x, y \in M$

$$\text{dist}(x, y) = \inf_{\Gamma} L(\gamma) \quad (2.4)$$

[AMS09] as the length of the shortest curve γ from the set Γ of all curves that pass x and y . Those length-minimizing shortest paths $\gamma(t)$ are called geodesics.



(a) Geodesics in the Euclidean plane $\mathbb{R}^2$

(b) Geodesics in the Lobachevsky plane $\mathbb{H}$

Figure 2.2.: Comparison of geodesics in different spaces, in the Euclidean plane they are straight lines, whereas in the Lobachevsky plane they are arcs of a circle.

2.1.3. The Exponential Map

During the optimization of the embeddings, data points in the embedding space are assigned to new points on the Riemannian manifold M . The direction and length of the

change depends on the Riemannian gradient $\text{grad } f$, which lies in the tangential space $T_p M$ of M at point p . The newly calculated value is assigned by the exponential map

$$\exp_p(v) := \gamma_{p,v}(1). \quad (2.5)$$

Geometrically speaking, it assigns a point $p \in M$ together with a tangent vector $v \in T_p M$ to a new point in M by evaluating the geodesic $\gamma(t)$ at $t = 1$ with initial velocity v at point p .

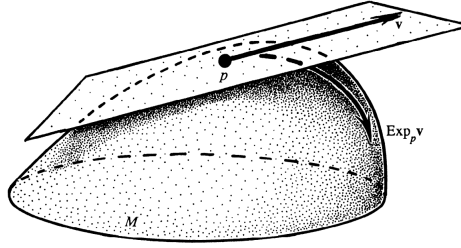


Figure 2.3.: The exponential map at point p [Mor93].

2.2. Numerical Optimization on Manifolds

The embedding of the data follows an optimization approach. This means, instead of directly calculating good representations of the data in the embedding space, they are first initialized and then continuously improved by minimizing a loss function $\mathcal{L}$. An overview of different initialization strategies is given in [Section 3.2.4](#). This chapter focuses on numerical optimization on manifolds in general, technical details will later be explained in [Section 3.2](#).

2.2.1. First Order Optimization

Before moving on to optimization on manifolds, consider the general optimization problem in the Euclidean space

$$\inf_{x \in \mathbb{R}^n} f(x), \quad f \in \mathcal{F} \quad (2.6)$$

i.e. the objective function f from the set of functions $\mathcal{F}$ is getting minimized by finding optimal values of x . This can be achieved by moving x in the direction of the steepest descent $-\text{grad } f(x(t))$ until a critical point with $\text{grad } f(x(t)) = 0$ is reached. A numerical

implementation is the gradient flow, which is the solution map $x_0 \rightarrow x(t)$ to the differential equation

$$\dot{x}(t) = -\text{grad } f(x(t)), \quad x(0) = x_0. \quad (2.7)$$

However, since this approach is impractical in most cases when using numerical methods [AMS09], the focus in the following is on algorithms

$$x_{t+1} = x_t + h_t \Delta x_t \quad (2.8)$$

which update x stepwise in the search direction $\Delta x_t \in \mathbb{R}^n$ with step size $h_t \in \mathbb{R}$. In practice, this can be implemented algorithmically in the following way:

Algorithm 1 Basic optimization algorithm

Require: x_0

while stopping criterion not achieved **do**

1. Determine the search direction Δx_t

2. Find a suitable step size h_t

▷ Line search

3. Update $x_{t+1} = x_t + h_t \Delta x_t$

end while

More detailed information on the line search step can be found in [Section 2.2.3](#).

2.2.2. Riemannian Stochastic Gradient Descent

The following section explains the Riemannian stochastic gradient descent algorithm as described in [NK17] and [Bon13], which present an iterative algorithm for optimization on Riemannian manifolds. From now on $\theta_t \in M$ will be used to denote points on the manifold, η_t the learning rate at step t and $\mathcal{L}$ a loss function, which is the objective function that is getting minimized.

Algorithm 2 Riemannian stochastic gradient descent (RSGD)

Require: $\theta_0 \in M$, loss function $\mathcal{L}$

while stopping criterion not satisfied **do**

1. Compute $\text{grad } \mathcal{L}(\theta_t)$

2. Find a suitable step size η_t

▷ Line search

3. Update $\theta_{t+1} = \exp_{\theta_t}(-\eta_t \text{grad } \mathcal{L}(\theta_t))$

end while

In [Algorithm 2](#), the Riemannian gradient of $\mathcal{L}$ is first computed. It is the tangent vector in $T_{\theta_t}M$ that describes the direction of the steepest ascent. It is then rescaled in length with a suitable step size η_t and reversed, such that it points in the direction of the steepest descent. To obtain the new θ_{t+1} the exponential map is used. Information on how this procedure is used in practice for the Poincaré disk is discussed in [Chapter 3](#).

2.2.3. Line Search - Accelerating RSGD

For fast convergence in RSGD, it's necessary to choose a suitable learning rate η_t in each step t . To ensure this, an adaptive learning rate is used during the optimization, which leads in practice to exponential convergence in the neighborhood of local minima [\[Sch23\]](#). The simplest assumption how we can choose η_t is

$$\mathcal{L}(\exp_{\theta_t}(-\eta_t \text{grad } \mathcal{L}(\theta_t))) < \mathcal{L}(\theta_t) \quad (2.9)$$

such that the loss decreases in each iteration. However, this condition is not enough to obtain convergence [\[NW06\]](#). Instead, a sufficient decrease condition is needed to guarantee convergence. A condition that ensures this is the Armijo condition

$$\mathcal{L}(\theta_t) - \mathcal{L}(\exp_{\theta_t}(-\eta_t \text{grad } \mathcal{L}(\theta_t))) \geq c \|\text{grad } \mathcal{L}(\theta_t)\|_g^2 \quad (2.10)$$

[\[Sch23\]](#) with $c > 0, \forall k \in \mathbb{N}$. Based on this condition, η_t is determined below in [Algorithm 3](#) using backtracking line search. This is an inexact line search algorithm that iteratively changes the step size until the Armijo condition is fulfilled. Inexact means that the optimal step size is not calculated, but an approximation that leads to a sufficient decrease, since calculating it exactly is too time-consuming in most cases.

Algorithm 3 Backtracking Line-Search

Require: $\theta_t \in M, \eta_t > 0, \tau, r \in (0, 1)$ (typical values : $\tau = \frac{1}{2}, r = 10^{-4}$)
while $\mathcal{L}(\theta_t) - \mathcal{L}(\exp_{\theta_t}(-\eta_t \text{grad } \mathcal{L}(\theta_t))) < r\eta_t \|\text{grad } \mathcal{L}(\theta_t)\|_g^2$ **do**
 Update step size $\eta_t = \tau\eta_t$
end while
return η_t

3. Poincaré Disk and Metric Data Embeddings

3.1. Hyperbolic Geometry

Hyperbolic geometry is a non-Euclidean geometry, which means that not all Euclidean axioms are fulfilled. To be precise, the fifth Euclidean axiom, which can be simpler expressed as the parallel postulate

”Given a line and a point not on it, there is exactly one line going through the given point that is parallel to the given line” [Can+97]

is not satisfied. Instead there can exist multiple lines through the given point which are parallel to the given line.

3.1.1. The Poincaré Disk

The model of hyperbolic geometry which will be used from now on is the Poincaré disk. It's the unit disk

$$\mathbb{D} = \{z \in \mathbb{C} : |z| < 1\} \quad (3.1)$$

with the associated Riemannian metric

$$ds = \frac{2|dz|}{1 - |z|^2}. \quad (3.2)$$

This forms a model of hyperbolic space. The boundary of the disk, which is the sphere S^1 , is denoted as $\partial\mathbb{D}$. It represents points that are infinitely distant away from the center [Can+97].

Geometric properties

The Riemannian metric ensures that lengths grow exponentially with increasing distance from the center. It follows that geodesics are only straight lines if they are diameters, otherwise they are an arc of the circle that is orthogonal to $\partial\mathbb{D}$.

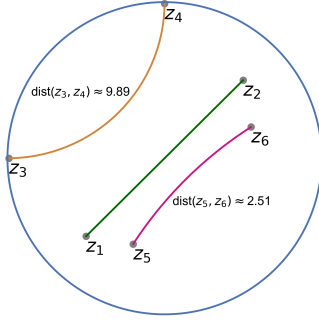


Figure 3.1.: Geodesics in the Poincaré disk.

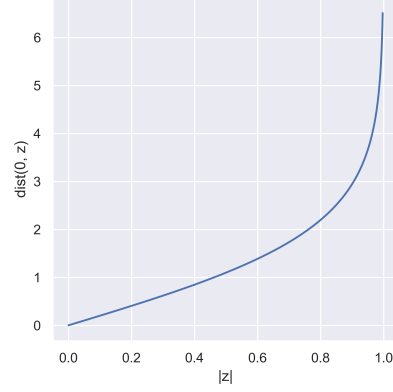


Figure 3.2.: Growth of Riemannian distance in Poincaré disk.

3.1.2. Riemannian distances in the Poincaré Disk

The Riemannian distance between two arbitrary points $z_1, z_2 \in \mathbb{D}$ can be computed directly using the cross ratio. However this is hard in practice, because it requires to find the points where the geodesic between z_1 and z_2 intersects $\partial\mathbb{D}$. Instead, a method is now derived where z_1 and z_2 are distance preserving transformed inside the Poincaré Disk, such that z_1 gets mapped to the center of $\mathbb{D}$. Then the Riemannian distance between z_1 and z_2 simplifies to the distance from the center of $\mathbb{D}$ to z_2 which is easy to compute.

Möbius Mappings

A Möbius mapping is a Riemannian isometrie, which results in the fact, that Riemannian distances are preserved [Can+97]. In general, Möbius mappings can be represented as

$$f(z) = e^{i\theta} \frac{a + z}{1 + \bar{a}z}. \quad (3.3)$$

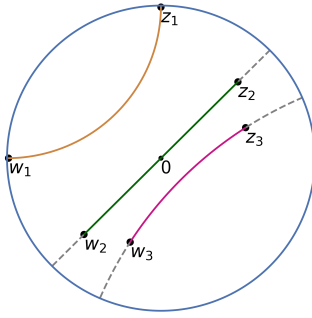
with $e^{i\theta} \in S^1$ and $a, z \in \mathbb{D}$.

Riemannian Distances in $\mathbb{D}$

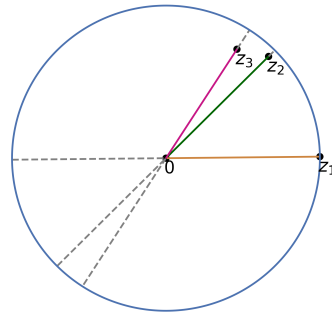
To derive the distance formula, the Möbius mapping

$$f(z) = \frac{z - w}{1 - \bar{w}z} \quad (3.4)$$

is used. For two points $z, w \in \mathbb{D}$, it maps w to the center of $\mathbb{D}$ and thereby ensures that the geodesic connecting z and w becomes a distance from the center in $\mathbb{D}$.



(a) Geodesics in $\mathbb{D}$.



(b) Geodesics after Möbius mapping.

The length of the geodesic can now be calculated using the Riemannian metric ds . This leads to the Riemannian distance formula in $\mathbb{D}$

$$d(z_1, z_2) = d(f(z_1), f(z_2)) = d(0, f(z_2)) \quad (3.5)$$

$$= \int_0^{f(z_2)} \frac{2dt}{1-t^2} \quad (3.6)$$

$$= \log \frac{1 + f(z_2)}{1 - f(z_2)} \quad (3.7)$$

$$= \log \frac{|1 - z_1 \bar{z}_2| + |z_1 - z_2|}{|1 - z_1 \bar{z}_2| - |z_1 - z_2|} \quad (3.8)$$

between two arbitrary points $z_1, z_2 \in \mathbb{D}$ [And05].

Exponential Map on the Poincaré Disk

The exponential map on the Poincaré Disk for $z \in \mathbb{D}$ with the associated tangent vector $v \in \mathbb{C}$ is denoted as

$$\exp_z(v) = z \oplus \left(\tanh\left(\frac{\lambda_z \|v\|}{2}\right) \frac{v}{\|v\|} \right) \quad (3.9)$$

[GBH18], with $\lambda_z := \frac{2}{1-\|z\|^2}$. Where $\oplus$ refers to the Möbius addition

$$u \oplus v = \frac{u + v}{1 + \bar{u}v} \quad (3.10)$$

[Ung01] for $u, v \in \mathbb{D}$.

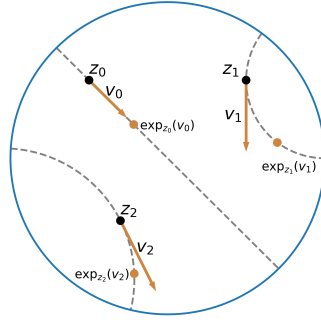


Figure 3.4.: Exponential maps on the Poincaré Disk.

3.2. Embedding Metric Data into the Poincaré Disk

3.2.1. Graph Data

The data that is embedded in the Poincaré disk has a graph structure. Formally a graph G is a tuple (V, E) consisting of a set of nodes (vertices) and a set of edges. This means that an edge $E \subseteq [V]^2$ represents the connection between two nodes in the graph. A path $P = (V, E)$ is a non-empty graph which shows the connection of two points in G . This leads to the distance $d_G(p_1, p_2)$ between two nodes $p_1, p_2 \in G$, as the length of the shortest path connecting p_1 and p_2 [Die17].

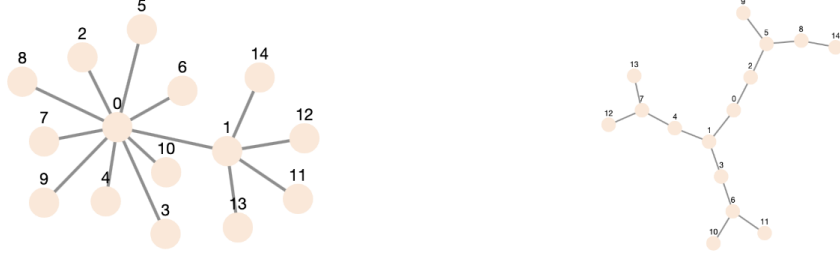


Figure 3.5.: Two graphs with 15 nodes. The left has a high branching factor b and the right a low branching factor b .

3.2.2. Loss Functions

Loss functions are used in the following as the objective function during the optimization process and to provide a way of measuring the quality of the embeddings. In the ongoing the average distortion [Sa+18] and the proposed loss from [ESY18] are chosen as loss functions. In the case of embedding data from the graph $G = (V, E)$ into the Poincaré Disk $\mathbb{D}$, which is represented as the set of n data points $\Theta = \{\vec{\theta}_i\}_{i=1}^n$, the average distortion results in

$$\mathcal{L}_{\text{dist}}(G, \Theta) = \frac{2}{n(n-1)} \sum_{\substack{p \neq q \in V \\ \vec{p} \neq \vec{q} \in \Theta}} \frac{|d_{\mathbb{D}}(\vec{p}, \vec{q}) - d_G(p, q)|}{d_G(p, q)} \quad (3.11)$$

with $p, q \in V$ and their corresponding representation in the Poincaré Disk $\vec{p}, \vec{q} \in \Theta$. The alternative loss function results in

$$\mathcal{L}_{\text{alt}}(G, \Theta) = - \sum_{p \in V} \sum_{q \in \mathcal{N}(p)} \log \frac{e^{-d_{\mathbb{D}}(\vec{p}, \vec{q})}}{\sum_{r \in \mathcal{N}^c(p)} e^{-d_{\mathbb{D}}(\vec{p}, \vec{r})}} \quad (3.12)$$

with $p, q, r \in V$ and their corresponding representations $\vec{p}, \vec{q}, \vec{r} \in \Theta$ in the Poincaré Disk. The neighbourhood $\mathcal{N}(p) = \{q \mid (p, q) \in E\}$, is the set containing all nodes, which are connected by an edge to p , all other nodes form the complement set $\mathcal{N}^c(p) = V \setminus \mathcal{N}(p)$.

Riemannian Gradient of Loss Functions

The Riemannian gradient of the loss function needs to be calculated in each optimization step. For that the Euclidean gradient of the loss function $\frac{\partial \mathcal{L}(G, \Theta)}{\partial \theta}$ is first calculated. In practice, the gradient of $\mathcal{L}$ depends on the chosen representation of the loss function, which results in

$$\frac{\partial \mathcal{L}_{\text{dist}}(G, \Theta)}{\partial \vec{\theta}} = \frac{2}{n(n-1)} \sum_{\substack{\vec{p} \in \Theta \\ \vec{p} \neq \vec{\theta}}} \frac{(d_{\mathbb{D}}(\vec{\theta}, \vec{p}) - d_G(\theta, p)) \cdot \frac{\partial d_{\mathbb{D}}(\vec{\theta}, \vec{p})}{\partial \vec{\theta}}}{d_G(\theta, p) \cdot |d_{\mathbb{D}}(\vec{\theta}, \vec{p}) - d_G(\theta, p)|} \quad (3.13)$$

for the average distortion (**Proof A**) and

$$\frac{\partial \mathcal{L}_{\text{alt}}(G, \Theta)}{\partial \vec{\theta}} = - \sum_{q \in \mathcal{N}(\theta)} - \frac{\partial d_{\mathbb{D}}(\vec{\theta}, \vec{q})}{\partial \vec{\theta}} - \frac{\sum_{r \in \mathcal{N}^c(p)} e^{-d_{\mathbb{D}}(\vec{\theta}, \vec{r})} \cdot - \frac{\partial d_{\mathbb{D}}(\vec{\theta}, \vec{r})}{\partial \vec{\theta}}}{\sum_{r \in \mathcal{N}^c(p)} e^{-d_{\mathbb{D}}(\vec{\theta}, \vec{r})}} \quad (3.14)$$

for the alternative loss function (**Proof A**). The corresponding partial derivative of the Riemannian distance in the Poincaré disk is

$$\frac{\partial d_{\mathbb{D}}(\vec{\theta}, x)}{\partial \vec{\theta}} = \frac{2 \left(|1 - \vec{\theta} \bar{x}|^2 (\vec{\theta} - x) - |\vec{\theta} - x|^2 (\bar{x}(\vec{\theta} \bar{x} - 1)) \right)}{|\vec{\theta} - x| |1 - \vec{\theta} \bar{x}| (|1 - \vec{\theta} \bar{x}| - |\vec{\theta} - x|) (|\vec{\theta} - x| + |1 - \vec{\theta} \bar{x}|)} \quad (3.15)$$

(**Proof B**). Since the length of vectors in Euclidean space differs from those in the Poincaré disk, they have to be rescaled with the inverse of the Riemannian metric tensor. This can be calculated trivially, because g_R results in a scalar matrix and leads to the final representation

$$\text{grad } \mathcal{L}(\vec{\theta}) = \frac{1 - |\vec{\theta}|^2}{2} \frac{\partial \mathcal{L}(\Theta)}{\partial \vec{\theta}} \quad (3.16)$$

of the Riemannian gradient $\text{grad } \mathcal{L}(\vec{\theta})$ of the loss function at the point $\vec{\theta} \in \Theta$.

3.2.3. Mini-Batch RSGD

The RSGD algorithm presented in **Algorithm 2** computes the gradient over the whole dataset, which may take a long time to calculate for large scale data. To get a good approximation of the gradient but still be tractable for large datasets, mini-batch RSGD is used. In mini-batch RSGD, the data is first split into random mini-batches, and then the gradient gets computed over the mini-batch instead of the whole data set [BB24]. In **Algorithm 4** θ_i represents the i -th embedding point at time t and N the total number of data points. The optimization runs until a sufficiently small value of the loss function is reached, this is referred to in **Algorithm 4** as the stopping criterion.

Algorithm 4 Mini-batch RSGD

Require: data points Θ_0 , loss function $\mathcal{L}$, batch size b

```

while stopping criterion not satisfied do
     $t \leftarrow 1$ 
     $i \leftarrow 1$ 
     $N \leftarrow |\Theta_t|$ 
    while  $i \leq N$  do
        1. Sample random mini-batch  $B$  from  $\Theta_t$  with size  $b$ 
        2.  $\theta_i \leftarrow \Theta_t[i]$ 
        3. Approximate  $\text{grad } \mathcal{L}(\theta_i)$  using  $B$ 
        4. Determine a step size  $\eta_i$  ▷ Line search
        5. Update  $\Theta_t[i] \leftarrow \exp_{\theta_i}(-\eta_i \text{grad } \mathcal{L}(\theta_i))$ 
        6.  $i \leftarrow i + 1$ 
    end while
     $t \leftarrow t + 1$ 
end while
    
```

3.2.4. Initialization Strategies

Random Initialization

One of the simplest and most efficient ways to initialize the data points in the Poincaré disk is to embed them randomly, such that the data points are sampled from a uniform distribution $\mathcal{U}$. In [NK17] it is suggested to do this near the origin of the disk using $\mathcal{U}(-0.001, 0.001)$, so that the points can be shifted further and further out during the optimization.

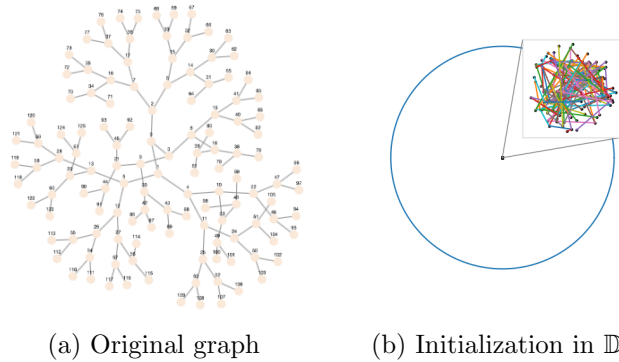


Figure 3.6.: A graph with 126 nodes and branching factor 2 is initialized into the Poincaré disk using random initialization with $\mathcal{U}(-0.001, 0.001)$.

MDS Initialization

A different approach, which may lead to faster convergence, is to compute an initialization that features a rough layout of the graph G with m nodes. To do so, good representations of G in the Euclidean plane are first computed using multidimensional scaling (MDS) [BG05]. For that, the euclidean distance matrix D , which is a symmetric matrix that holds

$$D_{ij} = d_G(p_i, p_j), \quad p_i, p_j \in G, \quad \forall i, j \in [m], \quad (3.17)$$

is first created. The MDS algorithm then uses D to find optimal representations $v_i \in \mathbb{R}^2$ which fulfill

$$\|\vec{v}_i - \vec{v}_j\|^2 \approx D_{ij}, \quad \vec{v}_1, \dots, \vec{v}_m \in \mathbb{R}^2, \quad \forall i, j \in [m]. \quad (3.18)$$

The data points must now be given a hyperbolic structure and be embedded in the Poincaré disk. The hyperbolic Lambert azimuthal projection [MKH22] can be used to give them the hyperbolic structure. Then the data points have to be mapped from the euclidean plane into the Poincaré disk. This can be done by directly scaling them into the disk using the mapping

$$\mathbb{R}^2 \rightarrow \mathbb{D}, \quad \vec{v}_i \mapsto \vec{\theta}_i = \frac{\vec{v}_i}{\max(V)} - \text{eps}, \quad V = \{\vec{v}_i\}_{i=1}^m, \quad (3.19)$$

where eps is a small constant that ensures $|\vec{\theta}_i| < 1$, $\forall i \in [m]$ after the scaling inside the disk.

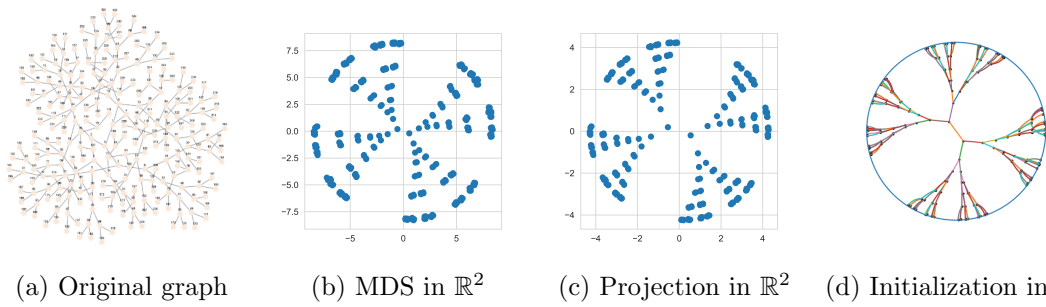


Figure 3.7.: A graph with 254 nodes and branching factor 2 is initialized into the Poincaré Disk using scaling MDS.

Another option is to first use a translation, such that all data points get mapped in the hyperbolic Lobachevsky plane $\mathbb{H} = \{z \in \mathbb{C} : \text{Im}(z) > 0\}$. They are then scaled, such that the center of the data is at the point $(0 + 1i) \in \mathbb{H}$.

$$T : \mathbb{R}^2 \rightarrow \mathbb{H}, \quad v \mapsto w = T(v) := v - (0 + \min(\text{Im}(V))) - \text{eps}, \quad V = \{\vec{v}_i\}_{i=1}^m, \quad (3.20)$$

$$S : \mathbb{H} \rightarrow \mathbb{H}, \quad w \mapsto z = S(w) := \left(\frac{2 \cdot \text{Re}(z)}{\max(\text{Re}(W))}, \frac{2 \cdot \text{Im}(z)}{\max(\text{Im}(W))} \right), \quad W = \{\vec{w}_i\}_{i=1}^m. \quad (3.21)$$

At the end they have to be transformed with the Cayley transformation

$$C : \mathbb{H} \rightarrow \mathbb{D}, \quad z \mapsto \vec{\theta} = C(z) := \frac{z - i}{z + i} \quad (3.22)$$

[Lee18] which maps a point $z \in \mathbb{H}$ into the Poincaré Disk $\mathbb{D}$. The scaling before the Cayley transformation ensures that the data points are transformed in the center of the disk.

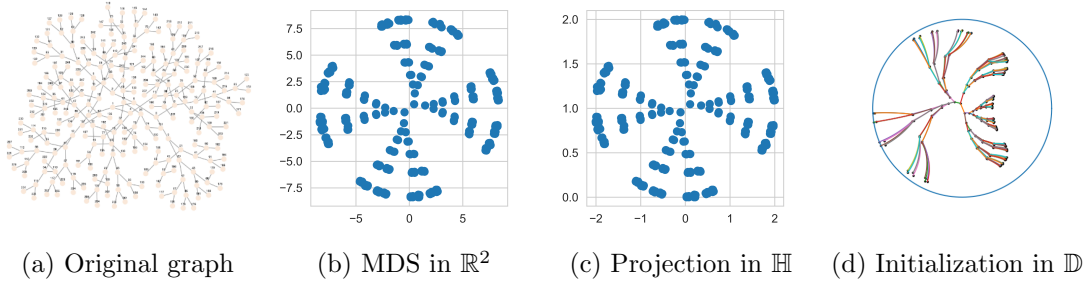


Figure 3.8.: A graph with 254 nodes and branching factor 2 is initialized in the Poincaré Disk using Cayley MDS.

4. Experiments and Discussion

4.1. Implementation Details

The whole implementation is written in the Python programming language, but all computationally intensive tasks have been outsourced to external libraries to improve performance. For all computations on graphs, the networkit library [SSM15] is used. In particular, the distance between two nodes in the graph is calculated using bidirectional Dijkstra, and the distance matrix D is generated using All-Pairs Shortest-Paths (APSP), which is also based on Dijkstra’s algorithm. All numerical operations are implemented strongly vectorized using the PyTorch library [Pas+19], because of its efficient implementation for all types of numerical computations. It also offers hardware acceleration on the GPU which is particularly important for large scale data. All gradients are computed with the PyTorch autograd feature. In order to ensure numerical stability, the Riemannian distance function was modified as in the implementation belonging to [NK17]. The random initialization is based on rejection sampling, and sklearn [Ped+18] is used for the MDS algorithm in the scaling and Cayley initialization. The following experiments were all carried out on an Apple M2 with 16GB of unified memory and double precision was used for all data types, which can be accessed in PyTorch using `torch.float64` for floating point numbers and `torch.complex128` for complex numbers.

4.2. Evaluation of Toy Data

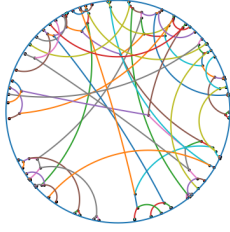
The performance of the loss functions is now evaluated using artificially generated graphs. These are created by sampling the number of children for each node from a Gaussian distribution, such that the distribution of nodes in the graph is represented according to this distribution. In all experiments, line search is used to select the step size.

4.2.1. Convergence Results of the Embedding Process

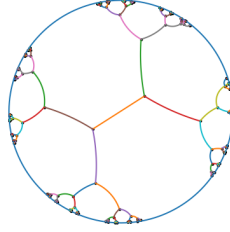
To gain comparability for the alternative loss function between graphs with different sizes, it was normalized in Table 4.1 with the number of edges in the graph.

Table 4.1.: Convergence results for graphs with branching factor 2 and different numbers of nodes. Each column in Table 4.1 represents the mean and the standard deviation of the embedding for 10 graphs with a fixed size to avoid outliers in the results. The bold numbers highlight the best result in each column for the average distortion and the alternative loss function. The average distortion is optimized for 1000 epochs whereas the alternative loss function is optimized for 200 epochs. The average distortion uses half of the number of nodes as batch size.

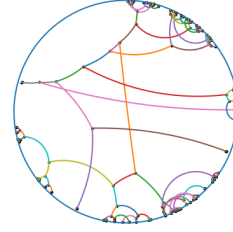
		Number of Nodes in G				
	Init	14	30	62	126	254
$\mathcal{L}_{\text{dist}}$	Uniform	0.10 (0.05)	0.21 (0.06)	0.41 (0.01)	0.40 (0.00)	0.79 (0.00)
	Scaling	0.09 (0.04)	0.09 (0.02)	0.08 (0.01)	0.07 (0.02)	0.06 (0.01)
	Cayley	0.07 (0.02)	0.06 (0.01)	0.08 (0.01)	0.12 (0.02)	0.18 (0.01)
$\mathcal{L}_{\text{alt}}$	Uniform	2.29 (0.07)	2.61 (0.12)	2.86 (0.12)	3.27 (0.15)	3.72 (0.13)
	Scaling	2.27 (7e-5)	2.47 (0.04)	2.57 (0.03)	2.64 (0.06)	2.71 (0.06)
	Cayley	2.27 (1e-4)	2.51 (0.08)	2.71 (0.07)	2.96 (0.08)	3.32 (0.10)



(a) Uniform initialization
($\mathcal{L}_{\text{alt}} = 3.72$)



(b) Scaling initialization
($\mathcal{L}_{\text{alt}} = 2.71$)



(c) Cayley initialization
($\mathcal{L}_{\text{alt}} = 3.32$)

Figure 4.1.: Convergence results for a graph with 254 nodes and branching factor 2 with different initializations using $\mathcal{L}_{\text{alt}}$.

The results in Table 4.1 show that as the number of nodes increases, the loss of the embeddings also increases, suggesting that larger graphs are generally more difficult to embed. Good results can nevertheless be achieved, even for large graphs, by choosing the right initialization. This is also illustrated in Figure 4.1, which concludes that the embedding using the scaling initialization outperforms the uniform and Cayley initializations. Another finding that emerges in the experiments with the alternative loss function is, that only a poor representation in the Poincaré disk is found for the

leaf nodes of the graph. This is clearly shown in Figure 4.4 in which the embeddings between the alternative loss function and the average distortion are very similar, but the representation of the leaf nodes in the graph is very different. The leaf nodes of the graph are always embedded very close to their corresponding parent node, which can be attributed to the fact that if a node has only one edge to another node, the alternative loss function gets minimized if the node is closer to its parent node.

Table 4.2.: Convergence results for a graph with 126 nodes and different branching factors following the Gaussian distribution $\mathcal{N}(\mu, \sigma^2)$. The bold numbers highlight the best result in each column for the average distortion and the alternative loss function.

		Distribution of Graph Creation			
Init		$\mu=5, \sigma^2=2.5$	$\mu=15, \sigma^2=7.5$	$\mu=25, \sigma^2=12.5$	$\mu=30, \sigma^2=15$
$\mathcal{L}_{\text{dist}}$	Uniform	0.31 (0.01)	0.32 (0.01)	0.34 (0.01)	0.33 (0.01)
	Scaling	0.14 (0.01)	0.20 (0.01)	0.21 (0.01)	0.22 (0.01)
	Cayley	0.15 (0.01)	0.20 (0.01)	0.21 (0.01)	0.21 (0.01)
$\mathcal{L}_{\text{alt}}$	Uniform	3.03 (0.05)	3.35 (0.08)	3.70 (0.16)	3.75 (0.13)
	Scaling	2.91 (0.06)	3.34 (0.08)	3.69 (0.16)	3.74 (0.13)
	Cayley	2.94 (0.07)	3.35 (0.08)	3.69 (0.16)	3.75 (0.13)

The results in Table 4.2 show that graphs with a higher branching factor are increasingly difficult to embed. It can also be concluded from the results that a good initialization becomes less relevant as the branching factor increases. This can be attributed to the fact that the scaling and Cayley initializations are based on finding good representations in $\mathbb{R}^2$ using MDS, which does not work well for graphs with high branching factors.

4.2.2. Runtime Efficiency of the Embedding Process

The runtime of the optimization is compared in Figure 4.2 between the embedding with average distortion or the alternative loss function. Again, the average distortion is trained with 1000 epochs, whereas the alternative loss function is trained for 200 epochs.

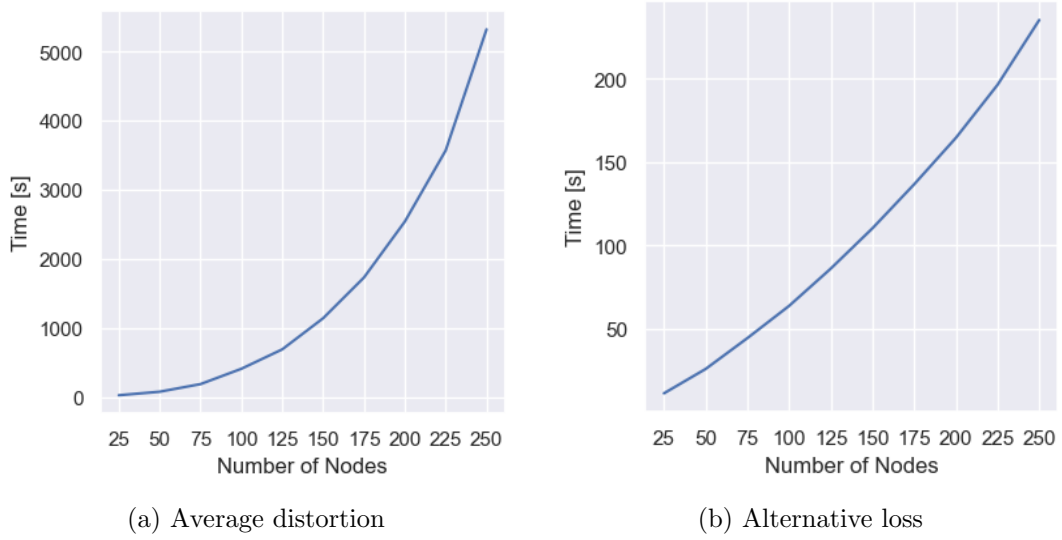


Figure 4.2.: Runtime of the embedding for a binary graph and scaling initialization.

The runtimes show that both loss functions have difficulties with larger graphs and already exhibit exponential growth. However, as the optimization currently only runs sequentially, this still offers many opportunities for parallelization. In [NK17] the parallelization of [Rec+11] is proposed, which makes it possible to efficiently embed even larger graphs, since a parallel approach can drastically improve the runtime with the appropriate hardware accelerator.

4.2.3. Comparing different Batch Sizes

When optimizing with the average distortion as the loss function, a mini-batch version is used. In Figure 4.3 the convergence results for a graph with branching factor 2 and 126 nodes are displayed, which is embedded using different batch sizes. Scaling initialization is used as initialization, as this shows the best results in Table 4.1.

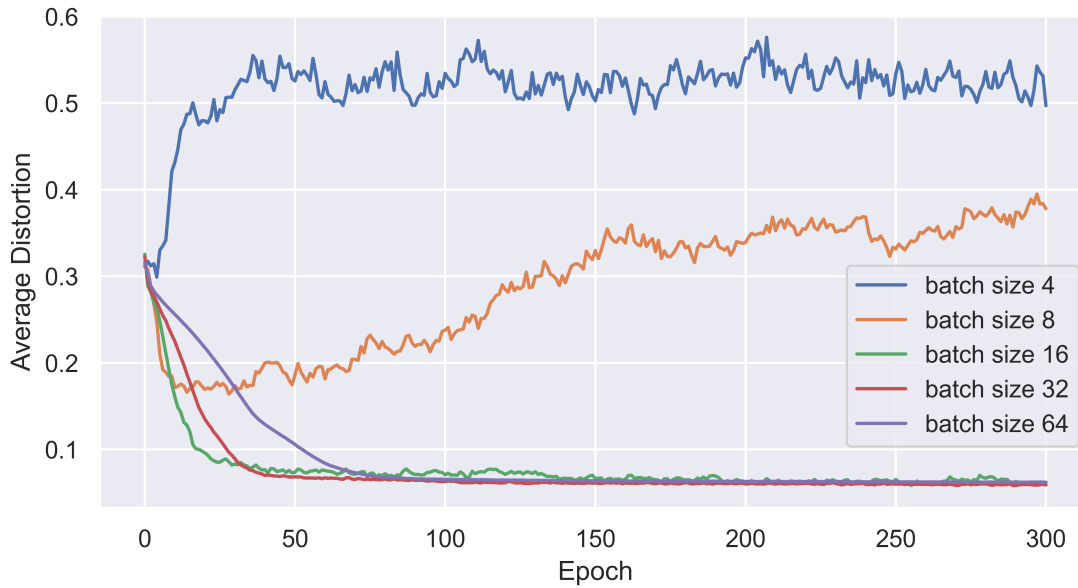


Figure 4.3.: Convergence results for different batch size.

The results in Figure 4.3 show that too small batch sizes (4 and 8) do not work, as they provide too inaccurate an approximation of the true gradient during the RSGD optimization. It can also be concluded that a smaller batch size can help to achieve significantly faster optimization. However, these then often fail when it comes to convergence.

4.3. Conclusion

The bachelor's thesis presented hyperbolic embeddings into the Poincaré disk from a solid mathematical background to experiments with toy data. These have shown that a basic optimization approach for simpler graphs already delivers good results. However, when it comes to larger and more complex graphs, the problems of poor convergence and long runtimes arise, whereby these can already be partially solved by choosing a good initialization and a suitable loss function. In conclusion, it can be said that hyperbolic data embeddings offer great potential for finding data representations, but the simple approach presented still needs to be adapted by making adjustments to the optimization and improving the runtime through parallelization, so that they can be applied to real world data.

4.4. Visualization of Exemplary Embeddings

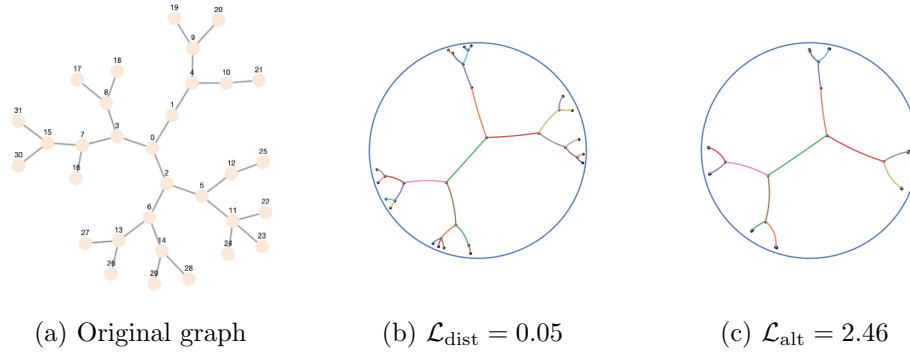


Figure 4.4.: Embedding of a graph with 32 nodes. The branching factors are sampled from $\mathcal{N}(2, 1)$.

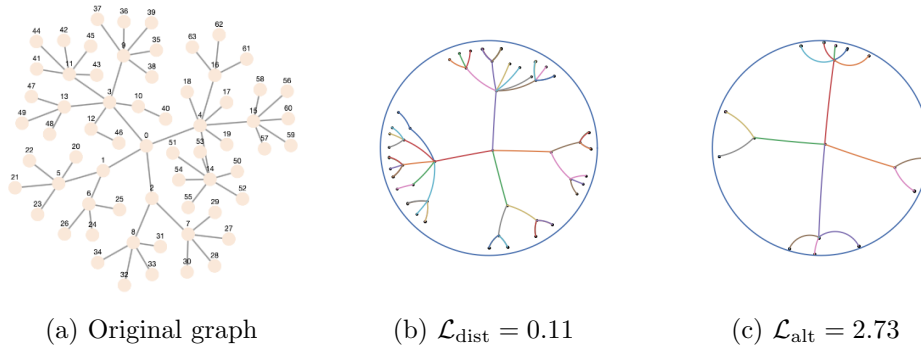


Figure 4.5.: Embedding of a graph with 64 nodes. The branching factors are sampled from $\mathcal{N}(3, 2)$.

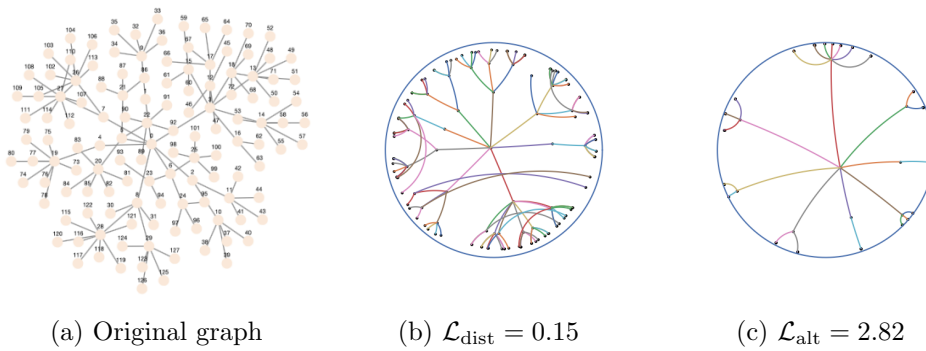


Figure 4.6.: Embedding of a graph with 128 nodes. The branching factors are sampled from $\mathcal{N}(4, 3)$.

Bibliography

- [Car92] M.P. do Carmo. *Riemannian Geometry*. Mathematics (Boston, Mass.) Birkhäuser, 1992. ISBN: 9783764334901. URL: <https://books.google.de/books?id=uXJQQgAACAAJ>.
- [Mor93] F. Morgan. *Riemannian Geometry: A Beginner's Guide*. Jones and Bartlett books in mathematics. Jones and Bartlett Publishers, 1993. ISBN: 9780867202427. URL: <https://books.google.lu/books?id=E17vAAAAMAAJ>.
- [Mil94] George A. Miller. “WordNet: A Lexical Database for English”. In: *Human Language Technology: Proceedings of a Workshop held at Plainsboro, New Jersey, March 8-11, 1994*. 1994. URL: <https://aclanthology.org/H94-1111>.
- [Can+97] J.W. Cannon et al. *Hyperbolic Geometry*. Vol. 31. Flavors of Geometry. MSRI Publications, 1997.
- [Ung01] A.A. Ungar. “Hyperbolic trigonometry and its application in the Poincaré ball model of hyperbolic geometry”. In: *Computers & Mathematics with Applications* 41.1 (2001), pp. 135–147. ISSN: 0898-1221. DOI: [https://doi.org/10.1016/S0898-1221\(01\)85012-4](https://doi.org/10.1016/S0898-1221(01)85012-4). URL: <https://www.sciencedirect.com/science/article/pii/S0898122101850124>.
- [And05] J. Anderson. *Hyperbolic Geometry*. Springer Undergraduate Mathematics Series. Springer London, 2005. ISBN: 9781852339340. URL: https://books.google.de/books?id=NYVnzAX8_qoC.
- [BG05] I. Borg and P.J.F. Groenen. *Modern Multidimensional Scaling: Theory and Applications*. Springer Series in Statistics. Springer New York, 2005. ISBN: 9780387251509. URL: <https://books.google.de/books?id=duTODldZzRcC>.
- [NW06] J. Nocedal and S. Wright. *Numerical Optimization*. Springer Series in Operations Research and Financial Engineering. Springer New York, 2006. ISBN: 9780387400655. URL: <https://books.google.de/books?id=VbHYoSyelFcC>.

- [AMS09] P.A. Absil, R. Mahony, and R. Sepulchre. *Optimization Algorithms on Matrix Manifolds*. Princeton University Press, 2009. ISBN: 9781400830244. URL: <https://books.google.de/books?id=NSQGQeLN3NcC>.
- [Rec+11] Benjamin Recht et al. “Hogwild!: A Lock-Free Approach to Parallelizing Stochastic Gradient Descent”. In: *Advances in Neural Information Processing Systems*. Ed. by J. Shawe-Taylor et al. Vol. 24. Curran Associates, Inc., 2011. URL: https://proceedings.neurips.cc/paper_files/paper/2011/file/218a0aefd1d1a4be65601cc6ddc1520e-Paper.pdf.
- [BCV12] Yoshua Bengio, Aaron C. Courville, and Pascal Vincent. “Representation Learning: A Review and New Perspectives”. In: *IEEE Transactions on Pattern Analysis and Machine Intelligence* 35 (2012), pp. 1798–1828. URL: <https://api.semanticscholar.org/CorpusID:393948>.
- [Bon13] Silvere Bonnabel. “Stochastic Gradient Descent on Riemannian Manifolds”. In: *IEEE Transactions on Automatic Control* 58.9 (Sept. 2013), pp. 2217–2229. ISSN: 1558-2523. DOI: [10.1109/tac.2013.2254619](https://doi.org/10.1109/tac.2013.2254619). URL: <http://dx.doi.org/10.1109/TAC.2013.2254619>.
- [NJT14] Maximilian Nickel, Xueyan Jiang, and Volker Tresp. “Reducing the Rank in Relational Factorization Models by Including Observable Patterns”. In: *Advances in Neural Information Processing Systems*. Ed. by Z. Ghahramani et al. Vol. 27. Curran Associates, Inc., 2014. URL: https://proceedings.neurips.cc/paper_files/paper/2014/file/556f391937dfd4398cbac35e050a2177-Paper.pdf.
- [SSM15] Christian L. Staudt, Aleksejs Sazonovs, and Henning Meyerhenke. *NetworKit: A Tool Suite for Large-scale Complex Network Analysis*. 2015. arXiv: [1403.3005](https://arxiv.org/abs/1403.3005) [cs.SI].
- [Car16] M.P. do Carmo. *Differential Geometry of Curves and Surfaces: Revised and Updated Second Edition*. Dover Books on Mathematics. Dover Publications, 2016. ISBN: 9780486806990. URL: <https://books.google.de/books?id=uXF6DQAAQBAJ>.
- [Die17] R. Diestel. *Graph Theory: 5th edition*. Springer Graduate Texts in Mathematics. Springer-Verlag, © Reinhard Diestel, 2017. ISBN: 9783961340057. URL: <https://books.google.de/books?id=zIxRDwAAQBAJ>.
- [NK17] Maximilian Nickel and Douwe Kiela. *Poincaré Embeddings for Learning Hierarchical Representations*. 2017. arXiv: [1705.08039](https://arxiv.org/abs/1705.08039) [cs.AI].

-
- [ESY18] Yosuke Enokida, Atsushi Suzuki, and Kenji Yamanishi. *Stable Geodesic Update on Hyperbolic Space and its Application to Poincare Embeddings*. 2018. arXiv: [1805.10487 \[stat.ML\]](#).
- [GBH18] Octavian Ganea, Gary Becigneul, and Thomas Hofmann. “Hyperbolic Neural Networks”. In: *Advances in Neural Information Processing Systems*. Ed. by S. Bengio et al. Vol. 31. Curran Associates, Inc., 2018.
- [Lee18] J.M. Lee. *Introduction to Riemannian Manifolds*. Graduate texts in mathematics. Springer, 2018. ISBN: 9783319917566. URL: <https://books.google.de/books?id=5RokxQEACAAJ>.
- [Ped+18] Fabian Pedregosa et al. *Scikit-learn: Machine Learning in Python*. 2018. arXiv: [1201.0490 \[cs.LG\]](#).
- [Sa+18] Christopher De Sa et al. *Representation Tradeoffs for Hyperbolic Embeddings*. 2018. arXiv: [1804.03329 \[cs.LG\]](#).
- [Pas+19] Adam Paszke et al. *PyTorch: An Imperative Style, High-Performance Deep Learning Library*. 2019. arXiv: [1912.01703 \[cs.LG\]](#).
- [MKH22] Jacob Miller, Stephen Kobourov, and Vahan Huroyan. *Browser-based Hyperbolic Visualization of Graphs*. 2022. arXiv: [2205.08028 \[cs.GR\]](#).
- [Sch23] Christoph Schnörr. *Numerical Optimization and Geometric Integration*. 2023.
- [BB24] Christopher M. Bishop and Hugh Bishop. *Deep Learning: Foundations and Concepts*. Springer, 2024.

Appendices

A. Euclidean Gradient of Loss Functions

Proof. Euclidean gradient of average distortion (3.13).

$$\frac{\partial \mathcal{L}_{\text{dist}}(G, \Theta)}{\partial \vec{\theta}} = \frac{\partial}{\partial \vec{\theta}} \left(\frac{2}{n(n-1)} \sum_{\substack{p \neq q \in V \\ \vec{p} \neq \vec{q} \in \Theta}} \frac{|d_{\mathbb{D}}(\vec{p}, \vec{q}) - d_G(p, q)|}{d_G(p, q)} \right) \quad (\text{A.1})$$

$$= \frac{2}{n(n-1)} \sum_{\substack{p \neq q \in V \\ \vec{p} \neq \vec{q} \in \Theta}} \frac{\partial}{\partial \vec{\theta}} \left(\frac{|d_{\mathbb{D}}(\vec{p}, \vec{q}) - d_G(p, q)|}{d_G(p, q)} \right) \quad (\text{A.2})$$

$$= \frac{2}{n(n-1)} \sum_{\substack{p \neq q \in V \\ \vec{p} \neq \vec{q} \in \Theta}} \frac{d_G(p, q) \cdot \left(\frac{(d_{\mathbb{D}}(\vec{p}, \vec{q}) - d_G(p, q)) \left(\frac{\partial d_{\mathbb{D}}(\vec{p}, \vec{q})}{\partial \vec{\theta}} - \frac{\partial d_G(p, q)}{\partial \vec{\theta}} \right)}{|d_{\mathbb{D}}(\vec{p}, \vec{q}) - d_G(p, q)|} \right)}{(d_G(p, q))^2} \quad (\text{A.3})$$

$$- \frac{|d_{\mathbb{D}}(\vec{p}, \vec{q}) - d_G(p, q)| \cdot \frac{\partial d_G(p, q)}{\partial \vec{\theta}}}{(d_G(p, q))^2} \quad (\text{A.4})$$

$$= \frac{2}{n(n-1)} \sum_{\substack{p \neq q \in V \\ \vec{p} \neq \vec{q} \in \Theta}} \frac{d_G(p, q) \cdot (d_{\mathbb{D}}(\vec{p}, \vec{q}) - d_G(p, q)) \cdot \frac{\partial d_{\mathbb{D}}(\vec{p}, \vec{q})}{\partial \vec{\theta}}}{(d_G(p, q))^2 \cdot |d_{\mathbb{D}}(\vec{p}, \vec{q}) - d_G(p, q)|} \quad (\text{A.5})$$

$$= \frac{2}{n(n-1)} \sum_{\substack{p \neq q \in V \\ \vec{p} \neq \vec{q} \in \Theta}} \frac{(d_{\mathbb{D}}(\vec{p}, \vec{q}) - d_G(p, q)) \cdot \frac{\partial d_{\mathbb{D}}(\vec{p}, \vec{q})}{\partial \vec{\theta}}}{d_G(p, q) \cdot |d_{\mathbb{D}}(\vec{p}, \vec{q}) - d_G(p, q)|} \quad (\text{A.6})$$

$$= \frac{2}{n(n-1)} \sum_{\substack{\vec{p} \in \Theta \\ \vec{p} \neq \vec{\theta}}} \frac{(d_{\mathbb{D}}(\vec{\theta}, \vec{p}) - d_G(\theta, p)) \cdot \frac{\partial d_{\mathbb{D}}(\vec{\theta}, \vec{p})}{\partial \vec{\theta}}}{d_G(\theta, p) \cdot |d_{\mathbb{D}}(\vec{\theta}, \vec{p}) - d_G(\theta, p)|} \quad (\text{A.7})$$

□

Proof. Euclidean gradient of alternative loss function (3.14).

$$\frac{\partial \mathcal{L}_{\text{alt}}(G, \Theta)}{\partial \vec{\theta}} = \frac{\partial}{\partial \vec{\theta}} \left(- \sum_{p \in V} \sum_{q \in \mathcal{N}(p)} \log \frac{e^{d_{\mathbb{D}}(\vec{p}, \vec{q})}}{\sum_{r \in \mathcal{N}^c(p)} e^{-d_{\mathbb{D}}(\vec{p}, \vec{r})}} \right) \quad (\text{A.8})$$

$$= - \sum_{p \in V} \sum_{q \in \mathcal{N}(p)} \frac{\partial}{\partial \vec{\theta}} \left(\log \frac{e^{-d_{\mathbb{D}}(\vec{p}, \vec{q})}}{\sum_{r \in \mathcal{N}^c(p)} e^{-d_{\mathbb{D}}(\vec{p}, \vec{r})}} \right) \quad (\text{A.9})$$

$$= - \sum_{p \in V} \sum_{q \in \mathcal{N}(p)} \frac{\partial}{\partial \vec{\theta}} \left(\log e^{-d_{\mathbb{D}}(\vec{p}, \vec{q})} \right) - \frac{\partial}{\partial \vec{\theta}} \left(\log \sum_{r \in \mathcal{N}^c(p)} e^{-d_{\mathbb{D}}(\vec{p}, \vec{r})} \right) \quad (\text{A.10})$$

$$= - \sum_{p \in V} \sum_{q \in \mathcal{N}(p)} \frac{\partial}{\partial \vec{\theta}} \left(-d_{\mathbb{D}}(\vec{p}, \vec{q}) \cdot \log e \right) - \frac{\frac{\partial}{\partial \vec{\theta}} \left(\sum_{r \in \mathcal{N}^c(p)} e^{-d_{\mathbb{D}}(\vec{p}, \vec{r})} \right)}{\sum_{r \in \mathcal{N}^c(p)} e^{-d_{\mathbb{D}}(\vec{p}, \vec{r})}} \quad (\text{A.11})$$

$$= - \sum_{p \in V} \sum_{q \in \mathcal{N}(p)} \frac{\partial}{\partial \vec{\theta}} \left(-d_{\mathbb{D}}(\vec{p}, \vec{q}) \right) - \frac{\sum_{r \in \mathcal{N}^c(p)} e^{-d_{\mathbb{D}}(\vec{p}, \vec{r})} \cdot \frac{\partial}{\partial \vec{\theta}} \left(-d_{\mathbb{D}}(\vec{p}, \vec{r}) \right)}{\sum_{r \in \mathcal{N}^c(p)} e^{-d_{\mathbb{D}}(\vec{p}, \vec{r})}} \quad (\text{A.12})$$

$$= - \sum_{p \in V} \sum_{q \in \mathcal{N}(p)} - \frac{\partial d_{\mathbb{D}}(\vec{p}, \vec{q})}{\partial \vec{\theta}} - \frac{\sum_{r \in \mathcal{N}^c(p)} e^{-d_{\mathbb{D}}(\vec{p}, \vec{r})} \cdot - \frac{\partial d_{\mathbb{D}}(\vec{p}, \vec{r})}{\partial \vec{\theta}}}{\sum_{r \in \mathcal{N}^c(p)} e^{-d_{\mathbb{D}}(\vec{p}, \vec{r})}} \quad (\text{A.13})$$

$$= - \sum_{q \in \mathcal{N}(\theta)} - \frac{\partial d_{\mathbb{D}}(\vec{\theta}, \vec{q})}{\partial \vec{\theta}} - \frac{\sum_{r \in \mathcal{N}^c(\theta)} e^{-d_{\mathbb{D}}(\vec{\theta}, \vec{r})} \cdot - \frac{\partial d_{\mathbb{D}}(\vec{\theta}, \vec{r})}{\partial \vec{\theta}}}{\sum_{r \in \mathcal{N}^c(\theta)} e^{-d_{\mathbb{D}}(\vec{\theta}, \vec{r})}} \quad (\text{A.14})$$

□

B. Partial Derivative of Riemannian Distance

Proof. Partial derivative of Riemannian distance of the Poincaré Disk (3.15).

$$\frac{\partial d_{\mathbb{D}}(\vec{\theta}, x)}{\partial \vec{\theta}} = \frac{\partial}{\partial \vec{\theta}} \left(\log \frac{|1 - \vec{\theta}\bar{x}| + |\vec{\theta} - x|}{|1 - \vec{\theta}\bar{x}| - |\vec{\theta} - x|} \right) \quad (\text{B.1})$$

$$= \frac{\partial}{\partial \vec{\theta}} \left(\log |1 - \vec{\theta}\bar{x}| + |\vec{\theta} - x| \right) - \frac{\partial}{\partial \vec{\theta}} \left(\log |1 - \vec{\theta}\bar{x}| - |\vec{\theta} - x| \right) \quad (\text{B.2})$$

$$= \frac{2 \left(|1 - \vec{\theta}\bar{x}| \cdot \frac{\partial |\vec{\theta} - x|}{\partial \vec{\theta}} - |\vec{\theta} - x| \cdot \frac{\partial |1 - \vec{\theta}\bar{x}|}{\partial \vec{\theta}} \right)}{|1 - \vec{\theta}\bar{x}|^2 - |\vec{\theta} - x|^2} \quad (\text{B.3})$$

$$= \frac{2 \left(|1 - \vec{\theta}\bar{x}| \cdot \frac{\vec{\theta} - x}{|\vec{\theta} - x|} - |\vec{\theta} - x| \cdot \frac{\bar{x}(\vec{\theta}\bar{x} - 1)}{|1 - \vec{\theta}\bar{x}|} \right)}{|1 - \vec{\theta}\bar{x}|^2 - |\vec{\theta} - x|^2} \quad (\text{B.4})$$

$$= \frac{2 \left(\frac{|1 - \vec{\theta}\bar{x}|(\vec{\theta} - x)}{|\vec{\theta} - x|} - \frac{|\vec{\theta} - x|(\bar{x}(\vec{\theta}\bar{x} - 1))}{|1 - \vec{\theta}\bar{x}|} \right)}{|1 - \vec{\theta}\bar{x}|^2 - |\vec{\theta} - x|^2} \quad (\text{B.5})$$

$$= \frac{2 \left(|1 - \vec{\theta}\bar{x}|(\vec{\theta} - x)|1 - \vec{\theta}\bar{x}| - |\vec{\theta} - x||\vec{\theta} - x|(\bar{x}(\vec{\theta}\bar{x} - 1)) \right)}{|\vec{\theta} - x||1 - \vec{\theta}\bar{x}|(|1 - \vec{\theta}\bar{x}| - |\vec{\theta} - x|)(|\vec{\theta} - x| + |1 - \vec{\theta}\bar{x}|)} \quad (\text{B.6})$$

$$= \frac{2 \left(|1 - \vec{\theta}\bar{x}|^2(\vec{\theta} - x) - |\vec{\theta} - x|^2(\bar{x}(\vec{\theta}\bar{x} - 1)) \right)}{|\vec{\theta} - x||1 - \vec{\theta}\bar{x}|(|1 - \vec{\theta}\bar{x}| - |\vec{\theta} - x|)(|\vec{\theta} - x| + |1 - \vec{\theta}\bar{x}|)} \quad (\text{B.7})$$

□